

KOENIG
step forward



Enterprise Java Full Stack Development



Programming in C



Language Introduction



Language Introduction

- Setting up the compiler (GCC) and the structure of a standard C program
- The compilation process: Source code to executable object file



Language Introduction

- C is a **general-purpose, procedural programming language** that was developed in the early **1970s** by **Dennis Ritchie** at **Bell Labs**.
- It was primarily designed for **system programming**, particularly for writing **operating systems** such as **UNIX**.
- Over time, C has become one of the most widely used and influential programming languages.



Language Introduction

- C is known for its **simplicity, efficiency, and flexibility**, making it a preferred choice for developing **system-level applications, embedded systems, and performance-critical software**.
- Many modern programming languages such as C++, Java, and C# have borrowed concepts from C.
- One of the key strengths of C is its ability to provide **low-level memory access** using pointers.
- This makes it powerful for developers who need control over hardware and memory management.



Language Introduction

- C follows a **procedural programming paradigm**, where programs are structured as a sequence of instructions organized into functions.
- It encourages **modular programming**, making code easier to **understand, maintain, and reuse**.



Language Introduction

- **Key Characteristics of C:**
- Simple and efficient syntax
- Structured (procedural) programming approach
- Rich set of operators and data types
- Direct memory access using pointers
- Portable across different platforms
- Fast execution and minimal runtime overhead



Language Introduction

- **Why Learn C?**
- Foundation for **learning other programming languages**
- Widely used in **system and embedded programming**
- Helps understand **core programming concepts** like memory, pointers, and data structures
- **High performance and close-to-hardware** capabilities



Setting up the compiler (GCC) and the structure of a standard C program

- GCC (GNU Compiler Collection) is one of the most widely used compilers for C programming.
- It converts C source code into executable programs.
- Installing GCC on Windows
 1. Install MSYS2
 2. Open MSYS2 terminal and run:

```
pacman -S mingw-w64-ucrt-x86_64-gcc
```
 3. Add the MinGW path to environment variables.
 4. Verify:

```
gcc --version
```



Setting up the compiler (GCC) and the structure of a standard C program

- Compiling and Running a C Program
 1. Write your program in a file (e.g., hello.c)
 2. Compile:

gcc hello.c -o hello

3. Run:

hello



Setting up the compiler (GCC) and the structure of a standard C program

- A C program follows a well-defined structure that improves **readability and maintainability**.

- Basic Structure

```
#include <stdio.h>    // Preprocessor directive

int main() {          // Main function (entry point)

    // Variable declaration
    int value = 100;

    // Statements
    printf("Value = %d\n", value);

    return 0;         // Exit status
}
```



Setting up the compiler (GCC) and the structure of a standard C program

- **Key Notes:**
- Every C program must have a **main()** function
- Statements end with a **semicolon (;)**
- C is **case-sensitive**
- Compilation happens before execution
- Errors must be fixed before running the program



The compilation process: Source code to executable object file

- When you write a C program, it goes through multiple stages before becoming a runnable executable.
- This process is handled by the compiler (GCC) and consists of five main steps:

Step 1: Source Code (.c file)

- This is the code written by the programmer.

Step 2: Preprocessing Stage

- Header files are included (#include)
- Macros are expanded (#define)
- Comments are removed



The compilation process: Source code to executable object file

Step 3: Compilation Stage

- Converts preprocessed code into assembly language
- Checks for syntax errors

Step 4: Assembly Stage

- Converts assembly code into machine code
- Generates object file

Step 5: Linking Stage

- Links object files with libraries (e.g., stdio)
- Resolves external references
- GCC performs all steps with a single command: `gcc hello.c -o hello`



Variables & Data Types



Variables & Data Types

- Primitive types: int, char, float, and double
- Formatted Input/Output using standard printf() and scanf() functions



Variables & Data Types

- A variable is a **named memory location** used to store data that can be modified during program execution.

- Declaration Syntax:

`data_type variable_name;`

- Example:

`int age = 25;`

`float salary = 50000.50;`

`char grade = 'A';`



Variables & Data Types

- **Rules for Naming Variables:**
- Must start with a letter or underscore (_)
- Cannot start with a number
- Cannot use C keywords (e.g., int, return)
- Case-sensitive (Age and age are different)



Primitive types: int, char, float, and double

- Data types define the type of data a variable can store and how much memory it occupies.
- Primitive (Basic) Data Types:
 - int
 - char
 - float
 - double



Primitive types: int, char, float, and double

- Integer Type (int)
 - Used to store whole numbers (no decimals).
 - Typically occupies 4 bytes
- Example:
 int count = 10;
 int temperature = -5;



Primitive types: int, char, float, and double

- Character Type (char)
 - Used to store a single character.
- Example:

```
char grade = 'A';  
char symbol = '#';
```
- Key Points:
 - Occupies 1 byte
 - Stored as ASCII value internally
 - Always enclosed in single quotes (' ')



Primitive types: int, char, float, and double

- Floating-Point Type (float)
 - Used to store decimal (fractional) numbers.
- Example:
float price = 99.99;
float temperature = 36.5;
- Key Points:
 - Typically occupies 4 bytes
 - Precision: ~6–7 decimal digits
 - Suffix f can be used: 99.99f



Primitive types: int, char, float, and double

- Double Type (double)
 - Used for high-precision decimal numbers.
- Example:
double pi = 3.1415926535;
double distance = 12345.6789;
- Key Points:
 - Typically occupies 8 bytes
 - Higher precision than float (~15 decimal digits)



Primitive types: int, char, float, and double

- Format Specifiers

Data Type		Format Specifier	
int		%d	
char		%c	
float		%f	
double		%lf	



Formatted Input/Output using standard printf() and scanf() functions

- Input and output operations in C are performed using standard library functions defined in the `<stdio.h>` header file.
- The most commonly used functions are `printf()` for output and `scanf()` for input.
- `printf()` – Formatted Output
 - `printf()` is used to display output on the console.
- Syntax:
`printf("format_string", value1, value2, ...);`



Formatted Input/Output using standard `printf()` and `scanf()` functions

- `scanf()` – Formatted Input
 - `scanf()` is used to read input from the user.
- Syntax:
`scanf("format_string", &variable);`
- Important Points about `scanf()`
 - Use **&** (address-of operator) for variables (except strings)
 - Input must match the format specifier
 - Values are separated by space, tab, or newline



Operators & Expressions



Operators & Expressions

- Arithmetic, relational, logical, and assignment operations
- Evaluating operator precedence and implicit/explicit type casting



Operators & Expressions

- **Operators** are symbols used to **perform operations** on variables and values.
- Example:
`int c = a + b; // '+' is an operator`
- An **expression** is a combination of variables, constants, and operators that **produces a value**.
- Example:
`int result = a + b * 2;`

Arithmetic, relational, logical, and assignment operations



- Arithmetic Operators
 - Used to perform mathematical calculations.

Operator	Description	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus (remainder)	$a \% b$

Arithmetic, relational, logical, and assignment operations



- Relational Operators
 - Used to compare two values.
 - The result is either true (1) or false (0).

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater or equal	a >= b
<=	Less or equal	a <= b



Arithmetic, relational, logical, and assignment operations

- Logical Operators
 - Used to combine multiple conditions.

Operator	Description	Example
&&	Logical AND	(a > 0 && b > 0)
 	Logical OR	(a > 0 b > 0)
!	Logical NOT	!(a > 0)



Arithmetic, relational, logical, and assignment operations

- Assignment Operators
 - Used to assign values to variables.

Operator	Description	Example
=	Assign	a = 10
+=	Add and assign	a += 5
-=	Subtract and assign	a -= 5
*=	Multiply and assign	a *= 2
/=	Divide and assign	a /= 2
%=	Modulus and assign	a %= 2

Evaluating operator precedence and implicit/explicit type casting

- Operator precedence determines the **order in which operations are evaluated** in an expression.

Level	Operators
1	(), [], ->, .
2	!, ++, --, unary + -
3	*, /, %
4	+, -
5	<, <=, >, >=
6	==, !=
7	&&
8	
9	=, +=, -=, etc.



Evaluating operator precedence and implicit/explicit type casting

- Type casting is converting one data type into another.
- There are two types:
 - Implicit (automatic)
 - Explicit (manual)
- Implicit Type Casting (Type Promotion)
 - Performed automatically by the compiler.
- Example:
 - `int a = 10;`
 - `float b = 2.5;`
 - `float result = a + b;` // a (int) is converted to float



Evaluating operator precedence and implicit/explicit type casting

- Explicit Type Casting
 - Manually converting a type using (type).
- Syntax:
(type) expression;
- Example:
int a = 5, b = 2;
float result = (float)a / b;



Control Flow & Functions



Conditional Statements



Conditional Statements

- Decision making using if, else if, else, and nested conditions
- Multi-way branching execution via the switch-case statement



Decision making using if, else if, else, and nested conditions

- Conditional statements allow a program to make decisions and execute different blocks of code based on certain conditions.
- **if Statement**
 - Executes a block of code only if the condition is true.

- **Syntax:**

```
if (condition) {  
    // code to execute if condition is true  
}
```

Example:

```
int age = 18;  
  
if (age >= 18) {  
    printf("Eligible to vote");  
}
```



Decision making using if, else if, else, and nested conditions

- **if-else Statement**

- Provides an alternative block if the condition is false.

- **Syntax:**

```
if (condition) {  
    // true block  
} else {  
    // false block  
}
```

Example:

```
int number = 5;  
  
if (number % 2 == 0) {  
    printf("Even");  
} else {  
    printf("Odd");  
}
```



Decision making using if, else if, else, and nested conditions

- **else if Ladder**

- Used when multiple conditions need to be checked.

- **Syntax:**

```
if (condition1) {  
    // block 1  
} else if (condition2) {  
    // block 2  
} else {  
    // default block  
}
```

Example:

```
int marks = 85;  
  
if (marks >= 90) {  
    printf("Grade A");  
} else if (marks >= 75) {  
    printf("Grade B");  
} else if (marks >= 50) {  
    printf("Grade C");  
} else {  
    printf("Fail");  
}
```



Decision making using if, else if, else, and nested conditions

- **Nested if Statements**

- An if statement inside another if statement.

- **Syntax:**

```
if (condition1) {  
    if (condition2) {  
        // code block  
    }  
}
```

Example:

```
int age = 20;  
int hasID = 1;  
if (age >= 18) {  
    if (hasID) {  
        printf("Allowed entry");  
    } else {  
        printf("ID required");  
    }  
} else {  
    printf("Underage");  
}
```



Decision making using if, else if, else, and nested conditions

- Logical Operators in Conditions
 - Conditions often use logical operators:
 - && (AND)
 - || (OR)
 - ! (NOT)
- Important Notes
 - Conditions evaluate to:
 - 1 → true
 - 0 → false
 - Use parentheses for clarity
 - Avoid deeply nested conditions (reduce complexity)
 - Always use braces {} for better readability

Multi-way branching execution via the switch-case statement



- The switch-case statement is used for **multi-way decision making**.
- It allows a program to execute one block of code among many options based on the value of an expression.
- It is an alternative to multiple **if-else if** statements when checking the same variable against different values.

Syntax

```
switch (expression) {  
    case value1:  
        // code block  
        break;  
  
    case value2:  
        // code block  
        break;  
  
    ...  
  
    default:  
        // default block (optional)  
}  

```

Multi-way branching execution via the switch-case statement



- **How It Works:**
- The **expression** is evaluated once
- Its value is compared with each **case**
- When a match is found, the corresponding block executes
- **break** exits the switch block
- **default** runs if no case matches



Looping Structures



Looping Structures

- Iteration logic with bounded for loops and conditional while loops
- Controlling loop execution paths with break and continue keywords



Iteration logic with bounded for loops and conditional while loops

- Looping structures allow a program to **repeat a block of code multiple times** based on a condition.
- They are essential for implementing iteration logic in C programs.



Iteration logic with bounded for loops and conditional while loops

- for Loop (Bounded Iteration)
 - The for loop is used when the number of iterations is known in advance.
- Syntax:

```
for (initialization; condition; update) {  
    // code to repeat  
}
```
- Flow:
 - Initialization → executed once
 - Condition → checked before each iteration
 - Update → executed after each iteration

Example:

```
#include <stdio.h>  
  
int main() {  
    for (int i = 1; i <= 5; i++) {  
        printf("%d\n", i);  
    }  
    return 0;  
}
```



Iteration logic with bounded for loops and conditional while loops

- while Loop (Conditional Iteration)
 - The while loop is used when the number of iterations is not known in advance and depends on a condition.

- Syntax:

```
while (condition) {  
    // code to repeat  
}
```

Example:

```
#include <stdio.h>  
  
int main() {  
    int i = 1;  
  
    while (i <= 5) {  
        printf("%d\n", i);  
        i++;  
    }  
  
    return 0;  
}
```



Controlling loop execution paths with break and continue keywords

- In C, loop execution can be controlled using special keywords like **break** and **continue**.
- These statements allow you to alter the normal flow of loops based on specific conditions.
- The break Statement
 - The break statement is used to terminate a loop immediately, even if the loop condition is still true.
- Syntax:
break;



Controlling loop execution paths with break and continue keywords

- The continue Statement
 - The continue statement is used to skip the current iteration and move to the next iteration of the loop.
- Syntax:
 continue;



Modular Functions



Modular Functions

- Defining functions, return types, function prototypes, and arguments
- Understanding scope differences: Local variables versus Global variables



Defining functions, return types, function prototypes, and arguments

- Modular programming means breaking a large program into smaller, manageable, reusable pieces called **functions**.
- Defining Functions
 - A function is a block of code that performs a specific task.
- Syntax:

```
return_type function_name(parameters) {  
    // function body  
}
```
- Function Arguments (Parameters)
 - Arguments are values passed to functions.

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```



Defining functions, return types, function prototypes, and arguments

- Return Types
 - The **return type** specifies what type of value the function will return.
- Function Prototypes
 - A **function prototype** tells the compiler about the function before it is used.
- Why needed?
 - Ensures correct function usage
 - Avoids compilation errors
 - Supports modular code (especially with multiple files)
- Syntax:
`return_type function_name(parameter_list);`



Understanding scope differences: Local variables versus Global variables

- Scope defines where a variable can be accessed in a program.
- There are two main types:
 - Local Variables
 - Global Variables
- Local Variables
 - A local variable is declared inside a function or block and is accessible only within that block.
 - Scope limited to function/block
 - Created when function is called
 - Destroyed when function exits



Understanding scope differences: Local variables versus Global variables

- Global Variables
 - A global variable is declared outside all functions and is accessible throughout the program.
 - Accessible in all functions
 - Lifetime = entire program execution
 - Default value = 0 (if not initialized)



Understanding scope differences: Local variables versus Global variables

- Global Variables
 - A global variable is declared outside all functions and is accessible throughout the program.
 - Accessible in all functions
 - Lifetime = entire program execution
 - Default value = 0 (if not initialized)



Data Structures & Pointers



Arrays & Matrices



Arrays & Matrices

- Declaring, initializing, and indexing single-dimensional arrays
- Multidimensional layouts: Creating and traversing 2D matrices



Declaring, initializing, and indexing single-dimensional arrays

- An **array** is a collection of elements of the same data type stored in **contiguous memory locations**.
- Declaring Arrays
- Syntax:
`data_type array_name[size];`
- Examples:
`int marks[5];`
`float prices[10];`
`char name[20];`
- This creates space for multiple values of the same type.



Declaring, initializing, and indexing single-dimensional arrays

- Initializing Arrays
 - Method 1: At Declaration

```
int arr[5] = {1, 2, 3, 4, 5};
```
 - Method 2: Partial Initialization

```
int arr[5] = {1, 2}; // Remaining elements = 0
```
 - Method 3: Size Not Specified

```
int arr[] = {10, 20, 30};
```
 - Method 4: Runtime Initialization

```
int arr[3];  
arr[0] = 5;  
arr[1] = 10;  
arr[2] = 15;
```



Declaring, initializing, and indexing single-dimensional arrays

- Indexing in Arrays
 - Array indexing starts from 0
 - Last index = size - 1
- Example:

```
int arr[3] = {10, 20, 30};
```

```
printf("%d", arr[0]); // 10
```

```
printf("%d", arr[2]); // 30
```



Multidimensional layouts: Creating and traversing 2D matrices

- A 2D array represents data in rows and columns (like a table or matrix).
- Think of it as an array of arrays.
- Declaring a 2D Array
- Syntax:
`data_type array_name[rows][columns];`
- Example:
`int matrix[3][4]; // 3 rows, 4 columns`



Multidimensional layouts: Creating and traversing 2D matrices

- Initializing a 2D Array

- Method 1: Full Initialization

```
int matrix[2][2] = {  
    {1, 2},  
    {3, 4}  
};
```

- Method 2: Inline Initialization

```
int matrix[2][2] = {1, 2, 3, 4};
```

- Method 3: Row Size Optional

```
int matrix[][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```



Multidimensional layouts: Creating and traversing 2D matrices

- Accessing Elements (Indexing)
- Syntax:
`matrix[row][column]`
- Example:
`printf("%d", matrix[1][0]); // Access row 1, column 0`
- Index starts from 0



Multidimensional layouts: Creating and traversing 2D matrices

- Accessing Elements (Indexing)
- Syntax:
`matrix[row][column]`
- Example:
`printf("%d", matrix[1][0]); // Access row 1, column 0`
 - Index starts from 0
- Common Matrix Operations
- Matrix Addition
`result[i][j] = A[i][j] + B[i][j];`
- Matrix Traversal (Print)
`printf("%d ", matrix[i][j]);`



Character Strings



Character Strings

- Managing text using null-terminated (`\0`) character arrays
- Core string operations utilizing standard `strlen`, `strcpy`, and `strcmp`



Managing text using null-terminated (\0) character arrays

- In C, a string is not a built-in type.
- It is represented as an array of characters ending with a null character '\0'.
- This null character marks the end of the string.
- **Declaring Strings**
- Syntax:
 `char str[size];`
- Example:
 `char name[10];`



Managing text using null-terminated (\0) character arrays

- Initializing Strings
- Method 1: Using String Literal
 - `char name[] = "John";`
 - Internally stored as: J o h n \0
- Method 2: Character Array
 - `char name[] = {'J', 'o', 'h', 'n', '\0'};`
- Method 3: Fixed Size
 - `char name[10] = "John";`



Managing text using null-terminated (\0) character arrays

- Understanding Null Terminator ('\0')
 - Marks end of string
 - ASCII value = 0
 - Used by all string functions
- Example:
`printf("%s", name);`
- Prints characters until '\0' is found



Managing text using null-terminated (\0) character arrays

- Accessing Characters

```
char str[] = "Hello";
```

```
printf("%c", str[0]); // H  
printf("%c", str[4]); // o
```

- Using scanf (single word only)

```
scanf("%s", str);
```

- Stops at space

- Using fgets (recommended)

```
fgets(str, sizeof(str), stdin);
```

- Reads full line (including spaces)

Core string operations utilizing standard strlen, strcpy, and strcmp



- To work with strings in C, include:
- `#include <string.h>`
- All these functions operate on null-terminated ('\0') character arrays.
- `strlen()` – Find String Length
 - Returns the number of characters excluding '\0'
 - Syntax: `size_t strlen(const char *str);`
- `strcpy()` – Copy Strings
 - Copies string from source → destination
 - Syntax: `char *strcpy(char *dest, const char *src);`

Core string operations utilizing standard strlen, strcpy, and strcmp



- strcmp() – Compare Strings
 - Compares two strings lexicographically
- Syntax:
`int strcmp(const char *s1, const char *s2);`
- Return Values:

Result	Meaning
0	Strings are equal
< 0	s1 < s2
> 0	s1 > s2



Pointers Basics



Pointers Basics

- Understanding memory addresses and using the reference operator (&)
- Declaring pointer variables and dereferencing them via the (*) operator



Understanding memory addresses and using the reference operator (&)

- Every variable in C is stored in memory and has a unique address.
- The **& operator** is called the address-of operator. It returns the address of a variable
- Syntax:

`&variable_name`



Declaring pointer variables and dereferencing them via the (*) operator

- A pointer is a variable that stores the memory address of another variable.
- Instead of holding a value, it holds where the value is stored.
- Pointer Declaration
- Syntax:
`data_type *pointer_name;`
- Examples:
`int *p; // Pointer to int`



Declaring pointer variables and dereferencing them via the (*) operator

- Assigning Address to Pointer

```
int x = 10;
```

```
int *p;
```

```
p = &x; // Assign address of x to pointer
```

- Now:

- $p \rightarrow$ holds address of x
- $*p \rightarrow$ gives value stored at that address

- Dereferencing Pointer (* Operator)

- The * operator is used to access or modify the value stored at the address.



Programming in C++



C++ Foundations & Syntax



Language Overview



Language Overview

- Setting up the C++ environment (GCC/Clang) and the main function structure
- Utilizing standard input/output streams via `std::cout` and `std::cin`



Language Overview

- C++ is a general-purpose, compiled programming language that supports both procedural and object-oriented programming (OOP).
- C++ is an extension of the C language.
- Where C++ is Used
 - Game engines (Unreal Engine)
 - Financial systems (low-latency trading)
 - Embedded systems
 - Operating systems
 - High-performance applications



Setting up the C++ environment (GCC/Clang) and the main function structure

- Understanding main() Function

- Syntax:

```
int main() {  
    // code  
    return 0;  
}
```

- Explanation:

- int → return type (integer)
- main() → entry point of program
- return 0 → indicates successful execution

Basic Structure:

```
// Header file for input/output  
#include <iostream>  
  
// Avoid writing std:: repeatedly  
using namespace std;  
  
int main() {  
    cout << "Hello, C++!";  
    return 0;  
}
```



Utilizing standard input/output streams via `std::cout` and `std::cin`

- C++ uses streams for input and output:
 - `std::cout` → Output stream (console output)
 - `std::cin` → Input stream (keyboard input)
- Both are part of the `<iostream>` library.



Utilizing standard input/output streams via `std::cout` and `std::cin`

- Output using `std::cout`

```
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "Hello, World!";
    return 0;
}
```

- `<<` is called the **insertion operator**



Utilizing standard input/output streams via `std::cout` and `std::cin`

- Input using `std::cin`
 `#include <iostream>`
 `using namespace std;`

```
int main() {  
    int x;  
    cin >> x;  
    cout << x;  
    return 0;  
}
```

- `>>` is called the **extraction operator**



Core Data Types & Control



Core Data Types & Control

- Declaring primitive types, working with `std::string`, and variable scoping
- Implementing conditional logic (if-else, switch) and iteration (for, while)



Declaring primitive types, working with `std::string`, and variable scoping

- C++ provides built-in (primitive) data types for storing values.
- Common Types

Type	Description	Example
int	Integer numbers	10
float	Decimal (low precision)	10.5
double	Decimal (high precision)	20.123456
char	Single character	'A'
bool	Boolean	true / false



Declaring primitive types, working with `std::string`, and variable scoping

- Working with `std::string`
 - Unlike C-style strings, C++ provides a powerful string class.

- Include Header

```
#include <string>
```

- Declaration and Usage

```
#include <iostream>
```

```
#include <string>
```

```
int main() {
```

```
    std::string name = "John";
```

```
    std::cout << name;
```

```
    return 0;
```

```
}
```



Declaring primitive types, working with `std::string`, and variable scoping

- Common String Operations

```
std::string first = "Hello";
```

```
std::string second = "World";
```

```
// Concatenation
```

```
std::string result = first + " " + second;
```

```
// Length
```

```
int len = result.length();
```

```
// Access character
```

```
char ch = result[0];
```



Declaring primitive types, working with `std::string`, and variable scoping

- Scope defines where a variable is accessible.
- Types of Scope

1. Local Scope

- Declared inside a function/block
- Accessible only within that block

```
void func() {  
    int x = 10; // local variable  
}
```

2. Global Scope

- Declared outside all functions
- Accessible throughout the program

```
int x = 100; // global  
  
int main() {  
    std::cout << x;  
}
```

3. Block Scope

```
if (true) {  
    int y = 20; // block scope  
}
```

100 // y is NOT accessible here

Implementing conditional logic (if-else, switch) and iteration (for, while)

- Conditional Statements (if, if-else, else-if)
 - Used to make decisions based on conditions.

- Basic if

```
int age = 20;
```

```
if (age >= 18) {  
    std::cout << "Eligible to vote";  
}
```

Implementing conditional logic (if-else, switch) and iteration (for, while)

- if-else

```
int num = 5;
```

```
if (num % 2 == 0) {  
    std::cout << "Even";  
} else {  
    std::cout << "Odd";  
}
```

Implementing conditional logic (if-else, switch) and iteration (for, while)

- else-if ladder

```
int marks = 75;
```

```
if (marks >= 90) {  
    std::cout << "Grade A";  
} else if (marks >= 70) {  
    std::cout << "Grade B";  
} else if (marks >= 50) {  
    std::cout << "Grade C";  
} else {  
    std::cout << "Fail";  
}
```

Implementing conditional logic (if-else, switch) and iteration (for, while)

- switch Statement

- Used when comparing a variable against multiple fixed values.

```
int day = 2;
```

```
switch(day) {
```

```
    case 1:
```

```
        std::cout << "Monday";
```

```
        break;
```

```
    case 2:
```

```
        std::cout << "Tuesday";
```

```
        break;
```

```
    case 3:
```

```
        std::cout << "Wednesday";
```

```
        break;
```

```
    default:
```

```
        std::cout << "Invalid day";
```

```
}
```


Implementing conditional logic (if-else, switch) and iteration (for, while)

- Iteration (Loops)
 - Loops are used to repeat a block of code.
- for Loop
 - Best when number of iterations is known.

```
for (int i = 1; i <= 5; i++) {  
    std::cout << i << " ";  
}
```

Implementing conditional logic (if-else, switch) and iteration (for, while)

- while Loop
 - Best when iterations depend on a condition.

```
int i = 1;
```

```
while (i <= 5) {  
    std::cout << i << " ";  
    i++;  
}
```

Implementing conditional logic (if-else, switch) and iteration (for, while)

- do-while Loop

- Executes at least once, even if condition is false.

```
int i = 1;
```

```
do {  
    std::cout << i << " ";  
    i++;  
} while (i <= 5);
```

Implementing conditional logic (if-else, switch) and iteration (for, while)

- Loop Control Statements

- break

- Stops the loop immediately
- ```
for (int i = 1; i <= 10; i++) {
 if (i == 5) break;
 std::cout << i;
}
```

- continue

- Skips current iteration
- ```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    std::cout << i;  
}
```



Object-Oriented Programming



Classes & Objects



Classes & Objects

- Defining classes, constructing objects, and managing access modifiers (public, private)
- Working with constructors, destructors, and initialization lists



Defining classes, constructing objects, and managing access modifiers (public, private)

- A class is a user-defined data type that groups:
 - Variables (data members)
 - Functions (member functions)
- It acts as a blueprint for creating objects.

Basic Class Definition

```
#include <iostream>
using namespace std;
class Student {
public:
    string name;
    int age;
    void display() {
        cout << "Name: " << name << ", Age: " << age;
    }
};
```




Defining classes, constructing objects, and managing access modifiers (public, private)

- An object is an instance of a class.

Creating and Using Objects

```
int main() {  
    Student s1;          // object creation  
  
    s1.name = "Ashok";  
    s1.age = 30;  
  
    s1.display();        // calling method  
    return 0;  
}
```



Defining classes, constructing objects, and managing access modifiers (public, private)

- Access modifiers control visibility of class members.
- Types of Access Modifiers

Modifier	Accessibility
public	Accessible everywhere
private	Accessible only inside the class
protected	Accessible in class + derived classes



Working with constructors, destructors, and initialization lists

- A constructor is a special member function:
 - Same name as class
 - No return type
 - Automatically called when an object is created
- Types of Constructors

1. Default Constructor

```
class Test {  
public:  
    Test() {  
        std::cout << "Default Constructor Called";  
    }  
};
```



Working with constructors, destructors, and initialization lists

2. Parameterized Constructor

```
class Student {  
public:  
    std::string name;  
    int age;  
  
    Student(std::string n, int a) {  
        name = n;  
        age = a;  
    }  
};
```



Working with constructors, destructors, and initialization lists

3. Constructor Overloading

```
class Demo {  
public:  
    Demo() {  
        std::cout << "No args";  
    }  
    Demo(int x) {  
        std::cout << "One arg: " << x;  
    }  
};
```



Working with constructors, destructors, and initialization lists

- Initialization Lists
 - Used to initialize variables before constructor body executes.

- Example

```
class Employee {  
private:  
    int id;  
    std::string name;  
public:  
    Employee(int i, std::string n) : id(i), name(n) {  
        // constructor body  
    }  
};
```



Working with constructors, destructors, and initialization lists

- Why Use Initialization List?
- Required for:
 - const variables
 - Base class initialization
 - Better performance (avoids extra assignment)



Working with constructors, destructors, and initialization lists

- A destructor:
 - Cleans up resources
 - Called automatically when object is destroyed
 - Same name as class with ~ prefix

- Example

```
class Test {  
public:  
    Test() {  
        std::cout << "Constructor called\n";  
    }  
};
```

```
~Test() {  
    std::cout << "Destructor called\n";  
}  
};
```

- When Destructor is Called
 - Object goes out of scope
 - Program ends
 - delete is used (for dynamic memory)



OOP Pillars in C++



OOP Pillars in C++

- Implementing single and multiple inheritance structures
- Achieving runtime polymorphism using virtual functions and abstract classes



OOP Pillars in C++

- C++ supports the four core pillars of OOP:
- Encapsulation → Data hiding using classes
- Abstraction → Hiding implementation details
- Inheritance → Reusing existing code
- Polymorphism → One interface, multiple behaviors



Implementing single and multiple inheritance structures

- Inheritance allows a class (**child/derived**) to acquire properties and behavior from another class (**parent/base**).
- Single Inheritance
 - One base class → One derived class
- Multiple Inheritance
 - One derived class inherits from multiple base classes
- Constructor Behavior in Inheritance
 - Base class constructor runs first
 - Then derived class constructor

Achieving runtime polymorphism using virtual functions and abstract classes



- Runtime polymorphism means:
 - The function call is resolved at runtime (late binding)
- Achieved using:
 - Virtual functions and Base class pointers/references
- Virtual Functions
 - A virtual function is a member function in a base class that is overridden in a derived class.
 - **virtual** keyword enables runtime binding
 - **override** ensures correct overriding
 - Base pointer → Derived object is essential

Achieving runtime polymorphism using virtual functions and abstract classes



- A pure virtual function (abstract class):

`virtual void func() = 0;`

- Makes the class abstract
- Cannot create object of abstract class
- Must override all pure virtual functions in derived class



Advanced Memory & Pointers



Pointers & References



Pointers & References

- Understanding memory addresses, pointer arithmetic, and dereferencing
- Using references (&) as aliases and passing parameters by reference



Understanding memory addresses, pointer arithmetic, and dereferencing

- A pointer is a variable that stores the memory address of another variable.
- Key Symbols
 - & → Address-of operator
 - * → Dereference operator
- Dereferencing a Pointer
- Dereferencing means accessing the value stored at the address

```
int x = 5;  
int* p = &x;  
cout << *p; // Output: 5
```



Understanding memory addresses, pointer arithmetic, and dereferencing

- Pointer Arithmetic
 - Pointers can be incremented/decremented.

Example

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {10, 20, 30};

    int* p = arr; // points to first
    element

    cout << *p << endl;      // 10
    cout << *(p + 1) << endl; // 20
    cout << *(p + 2) << endl; // 30
}
```



Using references (&) as aliases and passing parameters by reference

- A reference is simply an **alias (another name)** for an existing variable.
- Once created, both names refer to the **same memory location**

- **Rules of References**

- Must be initialized at declaration
- Cannot be NULL
- Cannot be reassigned to another variable
- Always refers to the same object

Basic Example

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int& ref = a;    // reference to a

    ref = 20;

    cout << "a = " << a << endl;    // 20
    cout << "ref = " << ref << endl; // 20
}
```



Using references (&) as aliases and passing parameters by reference

- Instead of copying values, we pass the actual variable

Example: Modify Value

```
#include <iostream>
using namespace std;

void update(int& x) {
    x = 100;
}

int main() {
    int a = 10;
    update(a);

    cout << a;    // Output: 100
}
```



Dynamic Memory Management



Dynamic Memory Management

- Allocating and deallocating memory manually via new and delete operators
- Modern memory management using smart pointers (`std::unique_ptr`, `std::shared_ptr`)



Allocating and deallocating memory manually via **new** and **delete** operators

- Dynamic Memory Management in C++ Using **new** and **delete**
- Dynamic memory is allocated at runtime instead of compile time.
- Useful when:
 - Size is unknown at compile time
 - You need flexible memory usage
 - Working with large data structures



Allocating and deallocating memory manually via `new` and `delete` operators

- Allocating Memory with **new**

- Single Variable

```
int* p = new int; // allocates memory
*p = 10;
```

- With Initialization

```
int* p = new int(25);
```

- Array Allocation

```
int* arr = new int[5]; // array of 5 integers
```



Allocating and deallocating memory manually via new and delete operators

- Deallocating Memory with **delete**
- Single Variable
`delete p;`
- Array
`delete[] arr;`
- Important Rule
 - Every new must have a corresponding delete



Modern memory management using smart pointers (`std::unique_ptr`, `std::shared_ptr`)

- Traditional `new` / `delete`:
 - Error-prone
 - Memory leaks
- Smart pointers solve this using RAII (Resource Acquisition Is Initialization)
 - Automatically free memory
 - Exception-safe
 - Cleaner code



Modern memory management using smart pointers (std::unique_ptr, std::shared_ptr)

- std::unique_ptr (Exclusive Ownership)
 - Only one pointer owns the object
- Key Features
 - Cannot be copied
 - Can be moved
 - Automatically deleted when out of scope

Basic Example

```
#include <iostream>
#include <memory>
using namespace std;

int main() {
    unique_ptr<int> p =
make_unique<int>(10);

    cout << *p << endl;    // 10
}
```



Modern memory management using smart pointers (std::unique_ptr, std::shared_ptr)

- std::shared_ptr (Shared Ownership)
 - Multiple pointers can share ownership
- Key Features
 - Reference counting
 - Memory freed when count = 0
 - Multiple owners allowed

Basic Example

```
#include <iostream>
#include <memory>
using namespace std;

int main() {
    shared_ptr<int> p1 =
make_shared<int>(100);
    shared_ptr<int> p2 = p1;

    cout << p1.use_count() << endl; //2
}
```



Java Programming



Java Fundamentals



Introduction & Environment Setup



Introduction & Environment Setup

- JDK, JRE, and JVM architecture
- Writing, compiling, and running a Java program



Introduction to Java

- Java is a **high-level, object-oriented, platform-independent** programming language developed by Sun Microsystems (now owned by Oracle).
- Java is widely used for building web, desktop, mobile, and enterprise applications.
- Key Features of Java
 - **Platform Independent** – “Write Once, Run Anywhere” (WORA) using JVM
 - **Object-Oriented** – Based on OOP concepts
 - **Simple & Secure** – No pointers, automatic memory management
 - **Robust** – Strong exception handling and type checking
 - **Multithreaded** – Supports concurrent execution
 - **Distributed** – Supports network-based applications

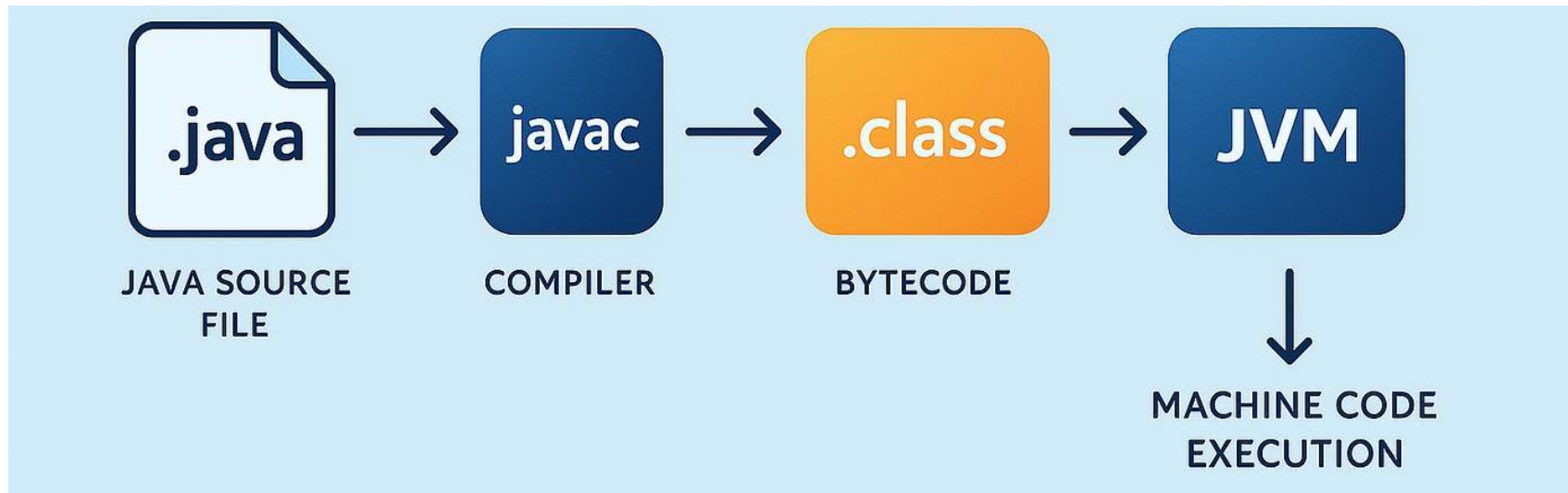


Setting up the development environment

- Install the Java Development Kit (JDK)
 - Download and Install JDK (latest LTS version recommended) from Oracle official site: <https://www.oracle.com/java/technologies/downloads/>
 - Set Environment Variables
 - Verify Installation: `java -version`
- Download and install IntelliJ IDEA (Recommended) from official site: <https://www.jetbrains.com/idea/>

JDK, JRE, and JVM architecture

- Java follows this execution flow:

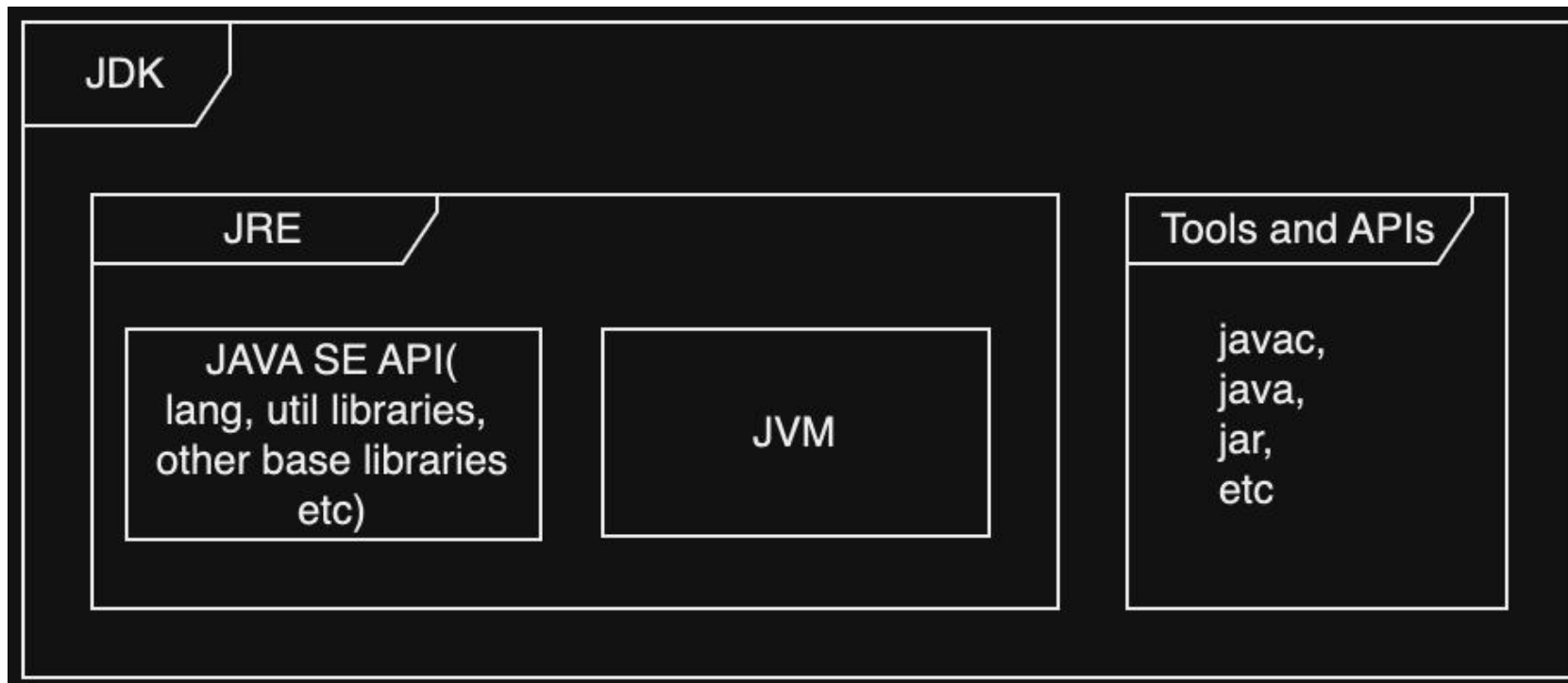


- JDK (Java Development Kit) – Development tools (compiler, debugger)
- JRE (Java Runtime Environment) – Libraries + JVM
- JVM (Java Virtual Machine) – Executes bytecode



JDK, JRE, and JVM architecture

- JDK (Java Development Kit)
 - The JDK is a complete toolkit for developing Java applications.





JDK, JRE, and JVM architecture

- The JVM is the core engine that executes Java bytecode and makes Java platform-independent.
- Key Components of JVM
 - Class Loader Subsystem
 - Loads .class files into memory, Phases: Loading → Linking → Initialization
 - Runtime Data Areas
 - Method Area – Class metadata, static variables
 - Heap – Objects
 - Stack – Method calls, local variables
 - PC Register – Current instruction pointer
 - Native Method Stack – For native code
 - Execution Engine
 - Interpreter – Executes bytecode line by line
 - JIT Compiler (Just-In-Time) – Converts bytecode to native machine code for performance
 - Garbage Collector (GC)
 - Automatically frees unused memory



JDK, JRE, and JVM architecture

- The JRE provides everything needed to run Java applications.
- Includes:
 - JVM
 - Core libraries (java.lang, java.util, etc.)
 - Supporting files
- JRE = JVM + Libraries



Writing, compiling, and running a Java program

- Writing Java Code Using IntelliJ IDEA
 - Creating a New Project
 - Open IntelliJ IDEA & Click on New Project.
 - Set a project name and location, then click Finish.
 - Adding a Java File
 - Right-click the src folder in your project.
 - Select New > Java Class and name it HelloWorld.
 - Writing the Code
 - Add the following code to the HelloWorld class
 - Running the Program
 - Right-click the file and select Run 'HelloWorld.main()'. - The output will appear in the Run console



Writing, compiling, and running a Java program

1. Write Code (HelloWorld.java)
 - You write Java source code using a text editor or IDE.
2. Compile Code (javac HelloWorld.java)
 - The Java Compiler (javac) translates the code into bytecode → HelloWorld.class
3. Load Class
 - The Class Loader loads the .class file into memory.
4. Verify & Prepare
 - Bytecode is verified for safety and memory is allocated for class variables.
5. Execute Code
 - The JVM executes the bytecode using the Execution Engine (interpreter + JIT compiler).



Language Basics



Language Basics

- Variables, data types, and operators
- Control flow statements (if-else, switch, loops)



Variables, data types, and operators

- A data type in Java defines the kind of data a variable can store, how much memory it uses, and the operations allowed on it.
- Data types ensure type safety during compilation.
- Data types help the compiler allocate memory efficiently.
- In Java, data types are categorized into two main types
 - Primitive Data Types (Built-in)
 - Reference Data Types



Variables, data types, and operators

- Primitive Data Types (Built-in)
 - These are the most basic data types provided by Java.
 - They are not objects and store actual values.

Type	Size	Description	Example
byte	1 byte	Whole number from -128 to 127	byte b = 100;
short	2 bytes	Whole number from -32K to 32K	short s = 30000;
int	4 bytes	Common integer type	int x = 100000;
long	8 bytes	Large integer values	long l = 123456L;
float	4 bytes	Decimal, single precision	float f = 3.14f;
double	8 bytes	Decimal, double precision	double d = 3.14159;
char	2 bytes	Single character (Unicode)	char c = 'A';
boolean	1 bit	True/False	boolean b = true;



Variables, data types, and operators

- Reference Data Types

- These types refer to objects, not actual values.
- Memory stores references (addresses) of the object.

Type	Description	Example
String	Represents a sequence of characters	<code>String s = "Hello";</code>
Arrays	Group of similar data elements	<code>int[] arr = {1,2,3};</code>
Classes	Blueprint for objects	<code>class Person {}</code> <code>Person p = new Person();</code>
Interfaces	Reference to a set of methods (contract)	<code>interface MyInterface {}</code> <code>class MyClass implements MyInterface {}</code> <code>MyClass obj = new MyClass();</code>



Variables, data types, and operators

- Variable Declaration
 - Declaring a variable means reserving memory space and specifying the data type.
- Syntax:
`dataType variableName;`
- Example:
`int age;     // declaration`
`String name; // declaration`



Variables, data types, and operators

- Variable Initialization
 - Initialization assign a value to the variable for the first time.
- Syntax:
 - `variableName = value;`
 - Or declare and initialize in one line:
`dataType variableName = value;`
- Example:
 - `int age = 25;       // declare + initialize`
 - `String name = "John"; // declare + initialize`



Variables, data types, and operators

- Arithmetic Operators
 - Used to perform basic mathematical operations.

Operator	Description	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2 (int)
%	Modulus (Remainder)	5 % 2	1



Variables, data types, and operators

- Relational (Comparison) Operators
 - Used to compare two values.

Operator	Description	Example	Result
==	Equal to	a == b	true / false
!=	Not equal to	a != b	true / false
>	Greater than	a > b	true / false
<	Less than	a < b	true / false
>=	Greater or equal	a >= b	true / false
<=	Less or equal	a <= b	true / false



Variables, data types, and operators

- Logical Operators
 - Used to combine multiple boolean expressions.

Operator	Description	Example	Result
&&	Logical AND	<code>a > 3 && b < 10</code>	true if both true
 	Logical OR	<code>a > 3 b < 10</code>	true if any one true
!	Logical NOT	<code>!(a > b)</code>	Opposite of condition



Variables, data types, and operators

- Unary Operators
 - Unary operators operate on a single operand.

Operator	Description	Example	Result
+	Unary plus	+a	a
-	Unary minus	-a	Negative a
++	Increment (prefix/postfix)	++a or a++	Increases a
--	Decrement (prefix/postfix)	--a or a--	Decreases a
!	Logical NOT	!true	false



Variables, data types, and operators

- Ternary Operator
 - The ternary operator is a shortcut for if-else and takes three operands.
- Syntax:
condition ? value_if_true : value_if_false;
- Example:
int a = 10, b = 20;
int max = (a > b) ? a : b;
System.out.println("Max = " + max); // Max = 20



Variables, data types, and operators

- Operator Precedence (Highest to Lowest)
 - Precedence determines which operator gets evaluated first when multiple operators are used in one expression.

Precedence	Operators	Type	Associativity
1	()	Parentheses	Left to right
2	++, --, +, -, !	Unary	Right to left
3	*, /, %	Arithmetic	Left to right
4	+, -	Arithmetic	Left to right
5	<, <=, >, >=	Relational	Left to right
6	==, !=	Equality	Left to right
7	&&	Logical AND	Left to right
8		Logical OR	Left to right
9	? :	Ternary	Right to left
10	=, +=, -=, *=, etc.	Assignment	Right to left

Control flow statements (if-else, switch, loops)

- if Statement
 - Used to execute a block of code only if a condition is true.
- Syntax:

```
if (condition) {  
    // code to execute if condition is true  
}
```

Control flow statements (if-else, switch, loops)

- if-else Statement
 - Used to execute one block if condition is true, otherwise another block.
- Syntax:

```
if (condition) {  
    // true block  
} else {  
    // false block  
}
```


Control flow statements (if-else, switch, loops)

- nested if-else Statement
 - Used when we have to make multiple decisions (conditions inside conditions).

- Syntax:

```
if (condition1) {  
    if (condition2) {  
        // inner true block  
    } else {  
        // inner false block  
    }  
} else {  
    // outer false block  
}
```

Control flow statements (if-else, switch, loops)

- switch-case Statement
 - Used to test a variable against multiple constant values.

- Syntax:

```
switch (variable) {  
    case value1:  
        // code block  
        break;  
    case value2:  
        // code block  
        break;  
    default:    // default code block  
}
```

Control flow statements (if-else, switch, loops)

- In Java, loops are used to execute a block of code repeatedly until a certain condition is met.

- for Loop Syntax:

```
for (initialization; condition; update) {  
    // loop body  
}
```

- while Loop

- Executes as long as the condition is true.

- Syntax:

```
while (condition) {  
    // loop body  
}
```

Control flow statements (if-else, switch, loops)

- do-while Loop
 - Executes at least once, even if the condition is false.
- Syntax:

```
do {  
    // loop body  
} while (condition);
```

Control flow statements (if-else, switch, loops)

- In Java, break, continue, and return are control flow keywords used to change the normal flow of program execution, especially inside loops and methods.
- **break** Keyword
 - Used to exit a loop or switch statement immediately.
- Syntax:
`break;`

Control flow statements (if-else, switch, loops)

- **continue** Keyword

- Used to skip the current iteration and continue with the next iteration of the loop.
- Syntax:

`continue;`

- **return** Keyword

- Used to exit from a method and optionally return a value.
- Syntax:

`return;           // in void methods`

`return value;    // in methods with return type`



Arrays & Strings



Arrays & Strings

- One-dimensional and multidimensional arrays
- String handling and core String methods

One-dimensional and multidimensional arrays



- **One-Dimensional (1D) Array**

- A 1D array is a list of elements of the same type arranged in a single row.
- Use 1D arrays for linear data (e.g., marks, ages).

- **Syntax:**

```
// Declaration
```

```
dataType[] arrayName;
```

```
// Declaration + Initialization
```

```
dataType[] arrayName = new dataType[size];
```

```
// Declaration + Initialization + Values
```

```
dataType[] arrayName = {val1, val2, val3};
```

One-dimensional and multidimensional arrays



- **One-Dimensional (1D) Array**

- Example:

```
int[] numbers = {10, 20, 30, 40};  
for (int i = 0; i < numbers.length; i++) {  
    System.out.println(numbers[i]);  
}
```

- Output:

```
10  
20  
30  
40
```

One-dimensional and multidimensional arrays



- **Multidimensional Arrays** (2D, 3D, etc.)
 - A multidimensional array is an array of arrays.
 - Use 2D or multi-D arrays for matrix-like or tabular data (e.g., table, matrix).
- **2D Array Syntax:**
 - // Declaration
dataType[][] arrayName;
 - // Declaration + Initialization
dataType[][] arrayName = new dataType[rows][columns];
 - // Declaration + Initialization + Values
dataType[][] matrix = { {1, 2, 3}, {4, 5, 6} };

One-dimensional and multidimensional arrays



- Multidimensional Arrays - Example:

```
int[][] matrix = { {1, 2}, {3, 4}, {5, 6} };  
for (int i = 0; i < matrix.length; i++) {  
    for (int j = 0; j < matrix[i].length; j++) {  
        System.out.print(matrix[i][j] + " ");  
    }  
    System.out.println();  
}
```

- Output:

```
1 2  
3 4  
5 6
```



String handling and core String methods

- In Java, **String** is one of the most commonly used classes, representing a sequence of characters.
- Strings in Java are immutable, meaning once created, they cannot be changed.
- Declaring and Initializing Strings
 - `String str1 = "Hello";` // String literal
 - `String str2 = new String("World");` // Using constructor



String handling and core String methods

- `length()`
 - Returns the number of characters in the string.
`String str = "Java";`
`System.out.println(str.length()); // Output: 4`
- `substring(int beginIndex, int endIndex)`
 - Extracts part of the string from `beginIndex` (inclusive) to `endIndex` (exclusive).
`String str = "Programming";`
`System.out.println(str.substring(0, 6)); // Output: Progra`
`System.out.println(str.substring(6)); // Output: mming`



String handling and core String methods

- String Comparison Methods
- equals(String str)
 - Compares content (case-sensitive).
`"Java".equals("java"); // false`
`"Java".equals("Java"); // true`
- equalsIgnoreCase(String str)
 - Compares content, ignoring case.
`"Java".equalsIgnoreCase("java"); // true`



String handling and core String methods

- Case Conversion Methods
- `toLowerCase()`
 - Converts all characters to lowercase.
`String s = "HeLLo";`
`System.out.println(s.toLowerCase()); // hello`
- `toUpperCase()`
 - Converts all characters to uppercase.
`String s = "HeLLo";`
`System.out.println(s.toUpperCase()); // HELLO`



String handling and core String methods

- Trimming Whitespace
- trim()
 - Removes leading and trailing whitespaces.
`String s = " Hello Java ";`
`System.out.println(s.trim()); // "Hello Java"`



Hands-on lab: Student mark Analysis (Using loops, arrays, and conditionals)

- Create a Java program that:
 - Accepts marks of N students.
 - Calculates the total, average, and highest mark.
 - Displays:
 - All students' marks and grades
 - Students who scored above average
 - The top scorer
 - Assigns grades based on the following:
 - Marks ≥ 90 → Grade A
 - Marks 80-89 → Grade B
 - Marks 70-79 → Grade C
 - Marks 60-69 → Grade D
 - Marks 50-59 → Grade E
 - Marks < 50 → Grade F



Object-Oriented Programming (OOP)



Classes & Objects



Classes & Objects

- Class definitions, object creation, and constructors
- Memory allocation and the 'this' keyword



Class definitions, object creation, and constructors

- Class
 - A class is a blueprint for creating objects.
 - It defines attributes (variables) and methods (functions).

```
public class Car {  
    // Attributes (fields)  
    String brand;  
    int speed;  
    // Method  
    void startEngine() {  
        System.out.println(brand + " engine started.");  
    }  
}
```



Class definitions, object creation, and constructors

- Object

- An object is an instance of a class.
- It represents a real-world entity with state and behavior.

```
public class Main {  
    public static void main(String[] args) {  
        Car car = new Car();    // Object creation  
        car.brand = "Honda";    // Accessing attribute  
        car.startEngine();      // Calling method  
    }  
}
```



Class definitions, object creation, and constructors

- Attributes

- Attributes (properties or fields or instance variables) define the state of an object.
- Example:

```
String brand;  
int speed;
```

- Methods

- Methods define the behavior of an object.
- They are like functions defined inside a class.
- Example:

```
void startEngine() {  
    System.out.println("Engine started");  
}
```




Class definitions, object creation, and constructors

- In Java, a **constructor** is a special method used to initialize objects.
- It has the same name as the class and no return type (not even void).
- Default Constructor
 - A default constructor is a constructor with no parameters.
 - If you don't define any constructor in a class, Java provides one automatically.

- Syntax:

```
public class Car {  
    Car() { // Default constructor  
        System.out.println("Car object created!");  
    }  
}
```



Class definitions, object creation, and constructors

- Parameterized Constructor
 - A parameterized constructor accepts arguments to initialize the object with specific values.

- Syntax:

```
public class Car {  
    String color;  
    Car(String c) { // Parameterized constructor  
        color = c;  
    }  
}
```



Memory allocation and the 'this' keyword

- **Garbage Collection** is an automatic memory management process in Java that reclaims memory used by objects no longer reachable.
- JVM removes unused objects to avoid memory leaks and optimize performance.
- **How It Works:**
 - When no references exist to an object, it becomes eligible for GC.
 - JVM's GC runs periodically to clean up such objects.



Memory allocation and the this keyword

- The '**this**' keyword in Java is a reference variable that refers to the current object of a class.
- Distinguish Between Instance and Local Variables
 - When local variables (e.g., constructor parameters) shadow instance variables, this helps refer to the current instance variable.

```
public class Student {  
    String name;  
    Student(String name) {  
        this.name = name; // 'this.name' refers to instance variable  
    }  
}
```



Memory allocation and the this keyword

- Invoke Current Class Constructor

- You can call another constructor in the same class using this().

```
public class Student {  
    String name;  
    int age;  
    Student() {  
        this("Unknown", 0); // Calls the parameterized constructor  
    }  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```



Memory allocation and the this keyword

- Access Current Class Methods and Variables
 - Even when there's no ambiguity, this can explicitly access members of the class.

```
public class Example {  
    void print() {  
        System.out.println("Print method");  
    }  
  
    void callPrint() {  
        this.print(); // Same as just print(), but explicit  
    }  
}
```



The Pillars of OOP



The Pillars of OOP

- Encapsulation and access modifiers
- Inheritance, polymorphism, and abstraction



Encapsulation and access modifiers

- **Encapsulation** is one of the four pillars of Object-Oriented Programming (OOP).
- It is the mechanism of wrapping data (variables) and code (methods) together into a single unit (a class) and restricting access to some of the object's components.
- Data hiding is the practice of restricting direct access to internal class variables.
- It's implemented using the private access modifier.
- The class exposes controlled access through getter and setter methods.



Encapsulation and access modifiers

- In Java, **access modifiers** define the scope (visibility) of classes, methods, constructors, and variables.
- They control where members of a class can be accessed from other classes.
- **public**
 - Access Level: Everywhere (same class, same package, outside package, subclass).
 - Use Case: When you want members to be accessible globally.

```
public class Car {  
    public String model = "Tesla";  
    public void start() {  
        System.out.println("Car started");  
    }  
}
```



Encapsulation and access modifiers

- **private**

- Access Level: Only within the **same class**.
- Use Case: When data/methods should not be accessed outside the class (encapsulation).

```
public class BankAccount {  
    private double balance;  
  
    private void calculateInterest() {  
        // only accessible inside this class  
    }  
}
```



Encapsulation and access modifiers

- **protected**

- Access Level: **Same class, Same package, Subclasses**
- Use Case: When members need to be visible to child classes or same package.

```
class Animal {  
    protected void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        eat(); // allowed due to protected  
    }  
}
```



Encapsulation and access modifiers

- **Default (no modifier)**

- Access Level: Package-private (accessible only within the same package).
- Use Case: When you want members accessible only to other classes in the same package.

```
class Book {  
    int pages = 100; // default access  
    void read() {  
        System.out.println("Reading book");  
    }  
}
```



Encapsulation and access modifiers

Modifier	Same Class	Same Package	Subclass	Other Classes
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
default	Yes	Yes	No	No
private	Yes	No	No	No



Inheritance, polymorphism, and abstraction

- **Inheritance** allows one class to inherit the properties and behaviors (fields and methods) of another class.
- It helps with code reusability and creating hierarchical relationships.
- Inheritance
 - Represented using extends keyword.
 - It defines a parent-child or superclass-subclass relationship.
 - It's also called class inheritance.



Inheritance, polymorphism, and abstraction

- **Inheritance** – Example

- Dog IS-A Animal → Because Dog inherits from Animal.

```
class Animal {  
    void eat() {  
        System.out.println("This animal eats food.");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("The dog barks.");  
    }  
}
```




Inheritance, polymorphism, and abstraction

- **Polymorphism** means many forms. In Java, it allows one interface to be used for a general class of actions. It comes in two types:
- Compile-Time Polymorphism (Static Binding)
 - Achieved by: Method Overloading
 - Method resolution happens at compile-time
 - Same method name, but different parameter types or counts
- Run-Time Polymorphism (Dynamic Binding)
 - Achieved by: Method Overriding
 - Method resolution happens at runtime
 - Involves inheritance and method overriding



Inheritance, polymorphism, and abstraction

- **Abstraction** is the process of hiding implementation details and showing only the essential features of an object.
- It helps reduce complexity by letting the user focus on what an object does instead of how it does it.

Benefit	Description
Hides details	Focus only on relevant behavior of objects
Improves reuse	Common behavior can be shared via abstract classes/interfaces
Promotes testing	Makes unit testing and mocking easier



Inheritance, polymorphism, and abstraction

- **Abstraction** Using Abstract Classes – Example

```
abstract class Animal {  
    abstract void sound(); // abstract method (no body)  
  
    void eat() {  
        System.out.println("This animal eats food.");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```



Inheritance, polymorphism, and abstraction

- **Abstraction** Using Interfaces – Example

```
interface Vehicle {  
    void start(); // public & abstract by default  
    void stop();  
}  
class Car implements Vehicle {  
    public void start() {  
        System.out.println("Car starts");  
    }  
    public void stop() {  
        System.out.println("Car stops");  
    }  
}
```



Inheritance, polymorphism, and abstraction

- Differences: **Abstract** Class vs Interface

Feature	Abstract Class	Interface
Methods	Can have both abstract and non-abstract	Only abstract methods (Java 7), Default & static (Java 8+)
Fields	Can have fields	Only constants (public static final)
Inheritance	Single inheritance	Multiple inheritance supported
Constructors	Can have constructors	Cannot have constructors
Access Modifiers	Can use all	All methods are public by default



Interfaces & Packages



Interfaces & Packages

- Defining and implementing interfaces
- Organizing code with packages



Defining and implementing interfaces

- In Java, **interfaces** allow abstraction and are the only way to achieve multiple inheritance of type.
- An interface is a reference type, similar to a class, that can contain:
 - Abstract methods (implicitly public and abstract)
 - Default and static methods (from Java 8)
 - Constants (public static final)

```
interface Animal {  
    void makeSound(); // abstract method  
}
```




Defining and implementing interfaces

- **Multiple Inheritance via Interface** // Multiple inheritance using interfaces

- Java supports multiple inheritance of behavior using interfaces

```
interface A {  
    void methodA();  
}  
  
interface B {  
    void methodB();  
}
```

```
class C implements A, B {  
    public void methodA() {  
        System.out.println("From A");  
    }  
    public void methodB() {  
        System.out.println("From B");  
    }  
}
```



Organizing code with packages

- **Packages** in Java are used to group related classes and interfaces and help avoid class name conflicts and control access.
- Create a User-Defined Package
 1. In the Project Explorer, right-click on src
 2. Click New > Package
 3. Enter package name: mypackage



Organizing code with packages

- Create a Java Class inside the Package

1. Right-click on the mypackage folder
2. Click New > Java Class
3. Name it: MyClass
4. Add the following code:

```
package mypackage;

public class MyClass {
    public void showMessage() {
        System.out.println("Hello from MyClass in mypackage!");
    }
}
```



Organizing code with packages

- Create the Main Class

1. Right-click on the src folder → New > Java Class
2. Name it: Main
3. Add this code:

```
import mypackage.MyClass;

public class Main {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.showMessage();
    }
}
```



Hands-on Lab: Bank Account Management

- Design a Java class called BankAccount to simulate basic banking operations.
 - Fields (set as private):
 - accountNumber (String), accountHolderName (String) and balance (double)
 - Implement the following:
 - A constructor to initialize all fields.
 - Getter methods to access the private fields.
 - A deposit(double amount) method to increase balance.
 - A withdraw(double amount) method to decrease balance only if sufficient funds are available.
 - A displayAccountInfo() method to print the account details.
 - In the BankDemo class:
 - Create at least two BankAccount objects.
 - Perform deposit and withdrawal operations.
 - Display updated account info.

Hands-on Lab: Employee Management System

- Design an Employee Management System using inheritance, method overloading, and method overriding.
 - Create a base class Employee with:
 - Fields: id, name, basicSalary
 - Constructor to initialize fields
 - A method calculateSalary() that returns the basicSalary
 - An overloaded method displayInfo():
 - One with no arguments
 - One with a boolean argument to hide/show salary
 - Create a subclass Manager that inherits from Employee:
 - Additional field: bonus
 - Override the calculateSalary() method to return basicSalary + bonus
 - Override the displayInfo() method to include manager's bonus
 - In the main class CompanyDemo, demonstrate:
 - Creating an Employee and a Manager
 - Calling both overloaded and overridden methods



Core Advanced Features



Exception Handling



Exception Handling

- try-catch-finally blocks
- Built-in and custom exceptions



What is Exception?

- An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions.
- Java provides a powerful exception-handling mechanism to gracefully manage errors like:
 - Division by zero
 - Accessing invalid array index
 - File not found
 - Null references, etc.



Types of exceptions – checked vs unchecked

- Java exceptions are mainly divided into two types:
- Checked Exceptions (Compile-Time Exceptions)
 - These are checked by the compiler at compile time.
 - The program won't compile unless these are either handled with try-catch or declared using throws.
- Examples:
 - IOException
 - FileNotFoundException
 - SQLException



Types of exceptions – checked vs unchecked

- Unchecked Exceptions (Runtime Exceptions)
 - These are not checked by the compiler.
 - Occur due to programming mistakes, like dividing by zero or null access.
 - You can handle them, but it's not mandatory.
- Examples:
 - ArithmeticException
 - NullPointerException
 - ArrayIndexOutOfBoundsException



try-catch-finally blocks

- The try-catch Block
 - In Java, the try-catch block is used to handle exceptions gracefully without crashing the program.

- Syntax:

```
try {  
    // Code that might throw an exception  
} catch (ExceptionType e) {  
    // Code to handle the exception  
}
```



try-catch-finally blocks

- Multiple Catch Blocks
 - You can handle different types of exceptions using multiple catch blocks.
 - Each block catches a specific exception.

- Syntax:

```
try {  
    // risky code  
} catch (ArithmeticException e) {  
    // handle arithmetic error  
} catch (ArrayIndexOutOfBoundsException e) {  
    // handle array index error  
} catch (Exception e) {  
    // handle all other exceptions  
}
```



try-catch-finally blocks

- Nested Try Blocks
 - A try block can be placed inside another try block. This is useful when different operations need separate exception handling.

- Syntax:

```
try {  
    // Outer try block  
    try {  
        // Inner try block  
    } catch (...) {  
        // Inner catch  
    }  
} catch (...) {  
    // Outer catch  
}
```



try-catch-finally blocks

- finally Block
 - The finally block is always executed after the try block (with or without a catch), regardless of whether an exception is thrown or not.

- Syntax:

```
try {  
    // risky code  
} catch (Exception e) {  
    // handle exception  
} finally {  
    // cleanup code that always runs  
}
```




try-catch-finally blocks

- throw Keyword

- The throw keyword is used to explicitly throw an exception (usually custom or runtime exception).
- Syntax:

```
throw new ExceptionType("message");
```

- throws Keyword

- The throws keyword is used in method declaration to indicate that the method might throw a checked exception, and the caller must handle it.
- Syntax:

```
public void readFile() throws IOException {  
    // code that might throw IOException  
}
```



try-catch-finally blocks

- Exception Propagation
 - When an exception occurs and is not caught in the current method, it propagates back to the caller method. This is known as exception propagation.
 - Only unchecked exceptions (like RuntimeException) propagate automatically. Checked exceptions require throws.



Built-in and custom exceptions

- A custom exception is a user-defined class that extends Java's `Exception` or `RuntimeException` classes to indicate specific error conditions.

- Extend `Exception` (Checked Exception)

```
public class InvalidAgeException extends Exception {  
    public InvalidAgeException(String message) {  
        super(message);  
    }  
}
```



Built-in and custom exceptions

- Extend RuntimeException (Unchecked Exception)

```
public class NegativeAmountException extends RuntimeException {  
    public NegativeAmountException(String message) {  
        super(message);  
    }  
}
```



Java Collections Framework



Java Collections Framework

- Lists (ArrayList, LinkedList)
- Sets (HashSet) and Maps (HashMap)



What is Collections Framework?

- The Java Collections Framework is a unified architecture for storing, retrieving, and manipulating groups of objects (data collections) efficiently.
- Key Components of the Collections Framework
 - Interfaces
 - Abstract data structures (e.g., List, Set, Map)
 - Implementations
 - Concrete classes (e.g., ArrayList, HashSet, LinkedList, HashMap)
 - Algorithms
 - Utility methods for searching, sorting, etc. (e.g., Collections.sort())



List, Set, Map – overview and differences

- List
 - A List is an ordered collection that allows duplicate elements.
- Key Features:
 - Maintains insertion order
 - Allows duplicate values
 - Access via index (0-based)
- Common Implementations:
 - ArrayList
 - LinkedList



List, Set, Map – overview and differences

- Set
 - A Set is a collection of unique elements.
- Key Features:
 - No duplicates allowed
 - No guaranteed order (unless using LinkedHashSet or TreeSet)
 - Cannot access elements via index
- Common Implementations:
 - HashSet – no order
 - LinkedHashSet – insertion order
 - TreeSet – sorted order



List, Set, Map – overview and differences

- Map
 - A Map stores key-value pairs.
- Key Features:
 - Unique keys
 - Values can be duplicated
 - Not a subtype of Collection
- Common Implementations:
 - HashMap – no order
 - LinkedHashMap – insertion order
 - TreeMap – sorted by keys



Working with ArrayList, LinkedList, HashSet, HashMap

- ArrayList – Dynamic Array

```
import java.util.ArrayList;
public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("Java"); // Allows duplicates
        System.out.println("ArrayList: " + list); // [Java, Python, Java]
    }
}
```

- Use case: Random access, frequent reads
- Backed by array → Fast for get(index)



Working with ArrayList, LinkedList, HashSet, HashMap

- LinkedList – Doubly Linked List

```
import java.util.LinkedList;
public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("C");
        list.add("C++");
        list.addFirst("Assembly");
        list.addLast("Java");
        System.out.println("LinkedList: " + list); // [Assembly, C, C++, Java]
    }
}
```

- Use case: Frequent insertion/removal
- Slower random access, but fast insertion/deletion



Working with ArrayList, LinkedList, HashSet, HashMap

- HashSet – Unique elements, no order

```
import java.util.HashSet;
public class HashSetDemo {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>();
        set.add("Apple");
        set.add("Banana");
        set.add("Apple"); // Duplicate ignored
        System.out.println("HashSet: " + set); // Output order not guaranteed
    }
}
```

- Use case: Store unique items
- Fast lookup, no duplicates



Working with ArrayList, LinkedList, HashSet, HashMap

- HashMap – Key-value pairs

```
import java.util.HashMap;
public class HashMapDemo {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(101, "Alice");
        map.put(102, "Bob");
        map.put(101, "Charlie"); // Replaces value for key 101
        System.out.println("HashMap: " + map); // {101=Charlie, 102=Bob}
        System.out.println("Value for key 102: " + map.get(102)); // Bob
    }
}
```

- Use case: Lookup by key
- Fast access, keys must be unique



Hands-on Lab: Library Book Management

- Create a system to manage library books where:
 - Each book has: bookId, title, author, and price.
 - You must:
 - Use ArrayList to store the books
 - Display the books



File I/O & Basic Streams



File I/O & Basic Streams

- Reading and writing files
- Introduction to Stream API basics



Reading and writing files

- Writing to File

```
try {  
    FileWriter fw = new FileWriter("example.txt");  
    fw.write("Hello from Java FileWriter!");  
    fw.close();  
    System.out.println("File written successfully.");  
}  
catch (IOException e) {  
    System.out.println(e);  
}
```



Reading and writing files

- Reading File

```
try {  
    FileReader fr = new FileReader("example.txt");  
    int ch;  
    while ((ch = fr.read()) != -1) {  
        System.out.print((char) ch);  
    }  
    fr.close();  
}  
catch (IOException e) {  
    System.out.println(e);  
}
```



Introduction to Stream API basics

- **Lambda Expressions (Java 8+)**

- A lambda expression is a short block of code that takes in parameters and returns a value.
- It can be used to implement methods of functional interfaces.

- **Syntax:**

(parameters) -> statement

or

(parameters) -> { statements }



Introduction to Stream API basics

- **Functional Interface**

- A Functional Interface in Java is an interface that contains exactly one abstract method.
- It can have any number of default or static methods.
- Functional Interfaces are used as the basis for lambda expressions and method references.



Introduction to Stream API basics

- Example: Creating a **Functional Interface**

```
@FunctionalInterface
```

```
interface MyFunctionalInterface {
```

```
    void doSomething(); // Single abstract method
```

```
}
```

- Usage with Lambda

```
MyFunctionalInterface obj = () -> System.out.println("Doing something!");
```

```
obj.doSomething();
```



Introduction to Stream API basics

- **Method References**

- Method references are a shorthand notation of a lambda expression to call a method.

- **Syntax:**

- `ClassName::methodName`

- **Example**

```
names.forEach(System.out::println);
```



Introduction to Stream API basics

- The **Stream API** (introduced in Java 8) is used to process collections of data (like List, Set, etc.) in a functional style using a sequence of operations like filtering, mapping, reducing, etc.
- Why use Stream API?
 - To perform bulk operations efficiently on collections
 - To write cleaner, more readable and declarative code
 - To enable pipelined processing (chaining operations)
 - To support parallel execution easily
- Stream Workflow
 - `collection.stream().filter(...).map(...).forEach(...);`



Introduction to Stream API basics

- `filter()`
 - Filters elements based on a given condition (Predicate).

```
List<String> names = List.of("Arun", "Anil", "Amit", "John");  
names.stream()  
    .filter(name -> name.startsWith("A"))  
    .forEach(System.out::println);
```



Introduction to Stream API basics

- `map()`
 - Transforms each element in the stream using a Function.

```
List<String> names = List.of("arun", "john");  
List<String> upper = names.stream()  
    .map(String::toUpperCase)  
    .collect(Collectors.toList());  
System.out.println(upper);
```



Introduction to Stream API basics

- `reduce()`
 - Reduces all elements in the stream to a single result (like sum).

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);  
int sum = numbers.stream()  
    .reduce(0, (a, b) -> a + b);  
System.out.println("Sum = " + sum);
```



Introduction to Stream API basics

- `forEach()`
 - Performs an action for each element (like printing).

```
List<String> cities = List.of("Delhi", "Mumbai", "Chennai");  
cities.stream().forEach(System.out::println);
```



Hands-on Lab: E-Commerce Orders System

- We have a list of Order objects.
- Use Stream API to:
 - Filter orders with amount > 1000
 - Convert customer names to uppercase
 - Calculate total order amount
 - Print each order
 - Collect high-value orders into a new list



Happy Learning :)